

Report 07

Latency Engineering & Performance Crisis Resolution

Project:	JaJeev — Agentic Marine Intelligence System
Phase:	Critical Engineering, Aug 27, 2026
Focus:	The latency crisis and the three rules that fixed it

1. The Crisis: 35 Seconds, Zero Ocean Data

On August 27, 2026, with all core agents wired into the LangGraph planner, we ran the first full end-to-end query and measured the result:

Metric	Before Fix
/marine/state (cold cache)	34.8 seconds
/marine/state (warm cache)	20.5 seconds
/chat	23.7 seconds
Ocean sources returned	0 — agent timed out

The last row was the emergency. The system was not merely slow — **it was returning a risk assessment computed with no ocean data at all. For a marine safety system, this is a correctness failure, not a performance problem.**

2. Root Cause Analysis

Three separate architectural mistakes compounded into a single failure:

2.1 Bug #1: Tide Was Cached Wrong

Tide was originally cached as "the height right now" — a single value with a short TTL (10 minutes). This forced a fresh expensive pyTMD computation on virtually every request.

Fix: Cache a **28-hour predicted tide series** instead. Astronomical tide is deterministic — a series computed this morning is still exactly correct this evening.

2.2 Bug #2: Cold Cache Blocked the Request

An uncached tide point caused the request to wait for the full computation (~8–12 seconds for pyTMD's harmonic model) before returning anything.

Fix: Return status: "computing" immediately and schedule the computation in a deduplicated background task.

2.3 Bug #3: One Slow Source Killed Everything

The `/marine/state` endpoint had a single `asyncio.gather()` with one wrapper timeout. Copernicus Marine's dataset-open call taking 15+ seconds discarded **all results**.

Fix: Per-source timeouts. Each adapter call is individually wrapped with `asyncio.wait_for(call, timeout=15)`.

2.4 Bug #4 (Gotcha): `asyncio.to_thread()` Contention

The LLM call (Groq API) was using `asyncio.to_thread()`, which shares asyncio's internal thread pool. Under load, the LLM call consistently timed out.

Fix: Route all blocking calls through the dedicated `network_executor` — a `ThreadPoolExecutor(max_workers=6)` never shared with asyncio internals.

3. The Three Rules (Now Permanent Architecture)

Rule 1: Cache What Is Time-Invariant, Not What Is Current

If a quantity can be predicted for a longer horizon (tide, sunrise/sunset, ephemeris), cache the full prediction and interpolate from it.

Rule 2: A Cold Cache Must Never Block a Request

If the cached value doesn't exist, return status: "computing" and kick off a background job. The next request benefits.

Rule 3: Bound the Expensive Resource, Not the Request

Per-source timeouts, never one timeout around a whole `gather()`. Blocking SDK calls go through a dedicated executor.

4. After-Fix Measurements

Endpoint	Before	After	Improvement
/marine/state (cold)	34.8s	2.5s	14× faster
/marine/state (warm)	20.5s	0.4s	51× faster
/chat	23.7s	4.3s	5.5× faster
/marine/quick-check (warm)	0.06s	0.06s	(already fast)
Ocean sources returned	0	5, including tide	■ Restored

5. The Copernicus Memory Problem (R-08)

`copernicusmarine.open_dataset()` materialises the chunk index of a *global* 1/12° hourly store. Peak RSS during a cold route query: **~7.2 GB**, which once killed the server process.

Partial Mitigations Applied

- Datasets are now closed promptly after subsetting (~1.2 GB)
- Concurrent dataset opens capped
- Prewarming via `scripts/prewarm_demo.py` makes the cold path rare

Not Yet Fixed

Opening each dataset once process-wide and subsetting the cached lazy object under a lock — this remains an open engineering risk (R-08).

6. The HDF5 Thread Safety Crash (R-09)

Symptom: The server process vanished mid-request. Nothing in stdout or stderr — no traceback, just gone.

Root cause: The HDF5 C library under netCDF4 is **not thread-safe**. The network_executor runs pyTMD, Copernicus, and erddapy concurrently, and they all read NetCDF/HDF5 files.

Mitigation: A process-wide netcdf_lock serialising all off-loop NetCDF/HDF5 reads was implemented. GEBCO bathymetry tiles are now eagerly loaded at startup.

Status: Implemented but **NOT verified under sustained load**. Must be confirmed before any demo.

7. The INCOIS ERDDAP SSL Issue (R-10)

During performance investigation, the INCOIS ERDDAP adapter was discovered to:

1. Fail with an SSL certificate-verification error on every call
2. Waste ~0.6 seconds per attempt (uncached)
3. Call griddap_initialize() on the event loop with no timeout

Mitigation: Wave height falls through to Open-Meteo Marine. Fix queued: move initialization to executor, add Redis cache, verify or disable the dataset ID.

8. The Prewarm Script

To avoid cold-cache latency during demos:

```
python scripts/prewarm_demo.py
```

Run 10–15 minutes before any demo. It:

1. Queries every demo coordinate point (Visakhapatnam, Puri, Kochi, Chennai)
2. Forces all adapters to populate Redis
3. Pre-computes 28-hour tide series for each point
4. Verifies successful cache hits for each variable